

Discrete Mathematics

CSE 4203

Dr. Md Azam Hossain

4.6 Cryptography

July 31, 2025

CHAPTER 4: Number Theory and Cryptography

- 4.1 Divisibility and Modular Arithmetic
- 4.2 Integer Representations and Algorithms
- 4.3 Primes and Greatest Common Divisors
- 4.4 Solving Congruences
- 4.5 Applications of Congruences
- 4.6 Cryptography

Background of Cryptography

- **Cryptography** is the study of techniques for securing communication and data.
- Originated thousands of years ago (e.g., Caesar cipher in ancient Rome).
- **Goal:** Transform information so that unauthorized users cannot understand it.
- **Two main branches:**
 - **Classical Cryptography:** Substitution and transposition ciphers using shared secret keys.
 - **Modern Cryptography:** Based on mathematical problems (e.g., factoring, discrete log), uses public/private keys.
- Key elements:
 - **Encryption:** Converting plaintext to ciphertext.
 - **Decryption:** Recovering plaintext from ciphertext.
 - **Key:** A secret value used in the encryption/decryption process.

4.6.2 Classical Cryptography

- **Classical cryptography** involves simple techniques to encrypt messages.
- Common in ancient and pre-modern times—used before the development of computers.
- Operates at the level of letters or blocks of letters.
- Examples include:
 - **Shift Cipher (Caesar Cipher)**: Each letter is replaced by another fixed number of positions later in the alphabet.
 - **Affine Cipher, Substitution Ciphers, Transposition Ciphers**
- **Private key cryptography**: Both sender and receiver use the same secret key for encryption and decryption.
- **Vulnerability**: Susceptible to **cryptanalysis**, which aims to break ciphers without the key.

Shift Cipher (Caesar Cipher)

- A **shift cipher** replaces each letter in the plaintext with a letter a fixed number of positions ahead in the alphabet.
- The key is the number of positions to shift.
- Letters are converted to numbers: A=0, B=1, ..., Z=25.

Encryption

$$E_k(x) = (x + k) \mod 26$$

Decryption

$$D_k(x) = (x - k) \mod 26$$

Example (Key $k = 3$)

- Plaintext: HELLO $\rightarrow$ H=7, E=4, L=11, L=11, O=14
- Encrypted: KHOOR $\rightarrow$ K=10, H=7, O=14, O=14, R=17

Cryptanalysis of Shift Ciphers

- Shift ciphers are easy to break using **brute-force attack**.
- There are only 26 possible keys ($k = 0$ to 25).
- Try all 26 shifts and check which one produces meaningful English text.

Cryptanalysis Strategy

- 1 Obtain the ciphertext.
- 2 For each key k , compute $D_k(x)$ for all characters.
- 3 Look for output that forms recognizable English words.

- **Limitation:** Effective only against simple ciphers.
- **Solution:** Use more secure systems like RSA.

Example 1: Caesar Cipher Encryption

Problem: Encrypt the message “MEET YOU IN THE PARK” using the Caesar cipher with key $k = 3$.

Step 1: Convert letters to numbers (A=0, ..., Z=25):

$$\text{MEET YOU IN THE PARK} \rightarrow \left\{ \begin{array}{l} M = 12, E = 4, E = 4, T = 19 \\ Y = 24, O = 14, U = 20 \\ I = 8, N = 13 \\ T = 19, H = 7, E = 4 \\ P = 15, A = 0, R = 17, K = 10 \end{array} \right.$$

Example 1: continue

Step 2: Apply encryption function $f(p) = (p + 3) \bmod 26$:

$$\left\{ \begin{array}{l} 12 + 3 = 15, \quad 4 + 3 = 7, \quad 4 + 3 = 7, \quad 19 + 3 = 22 \\ 24 + 3 = 27 \equiv 1, \quad 14 + 3 = 17, \quad 20 + 3 = 23 \\ 8 + 3 = 11, \quad 13 + 3 = 16 \\ 19 + 3 = 22, \quad 7 + 3 = 10, \quad 4 + 3 = 7 \\ 15 + 3 = 18, \quad 0 + 3 = 3, \quad 17 + 3 = 20, \quad 10 + 3 = 13 \end{array} \right.$$

Encrypted message numbers: 15 7 7 22 1 17 23 11 16 22 10 7 18 3 20 13

Encrypted message: **“PHHW BRX LQ WKH SDUN.”**

Affine Cipher

- The **Affine Cipher** is a generalization of the shift cipher.
- Encryption uses both multiplication and addition modulo 26.
- Each letter is converted to a number: $A = 0, B = 1, \dots, Z = 25$.

Encryption Function

$$E_{a,b}(x) = (a \cdot x + b) \mod 26$$

Decryption Function

$$D_{a,b}(x) = a^{-1} \cdot (x - b) \mod 26$$

Where a^{-1} is the modular inverse of a modulo 26.

- a must be coprime to 26 for the cipher to be reversible.
- Common valid a values: 1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25.

Example: Affine Cipher ($a = 5$, $b = 8$)

- Plaintext: HELP
- Convert to numbers: $H = 7$, $E = 4$, $L = 11$, $P = 15$

Encryption Function

$$E_{5,8}(x) = (5 \cdot x + 8) \mod 26$$

- H (7): $(5 \cdot 7 + 8) \mod 26 = 43 \mod 26 = 17 \rightarrow R$
- E (4): $(5 \cdot 4 + 8) \mod 26 = 28 \mod 26 = 2 \rightarrow C$
- L (11): $(5 \cdot 11 + 8) \mod 26 = 63 \mod 26 = 11 \rightarrow L$
- P (15): $(5 \cdot 15 + 8) \mod 26 = 83 \mod 26 = 5 \rightarrow F$

Ciphertext: RCLF

Decryption

$$a^{-1} = 21 \quad (\text{since } 5 \cdot 21 \equiv 1 \mod 26)$$

$$D_{5,8}(x) = 21(x - 8) \mod 26$$

Affine Cipher Decryption ($a = 5$, $b = 8$, $a^{-1} = 21$)

- Ciphertext: RCLF
- Convert to numbers: $R = 17$, $C = 2$, $L = 11$, $F = 5$

Decryption Function

$$D_{5,8}(x) = 21(x - 8) \mod 26$$

- $R (17)$: $21(17 - 8) = 21 \cdot 9 = 189 \mod 26 = 7 \rightarrow H$
- $C (2)$: $21(2 - 8) = 21 \cdot (-6) = -126 \mod 26 = 4 \rightarrow E$
- $L (11)$: $21(11 - 8) = 21 \cdot 3 = 63 \mod 26 = 11 \rightarrow L$
- $F (5)$: $21(5 - 8) = 21 \cdot (-3) = -63 \mod 26 = 15 \rightarrow P$

Decrypted Plaintext: HELP

Cryptanalysis is the process of recovering the plaintext from ciphertext *without knowing* the encryption method or key.

Example – Shift Cipher Attack:

- If the encryption method is a shift cipher (Caesar cipher), we exploit the fact that there are only **26 possible shifts**.
- To recover the plaintext:
 - Try all possible shifts (from $k = 0$ to $k = 25$).
 - Decrypt the ciphertext for each shift.
 - One result will be readable plaintext.
- This method is called a **brute-force attack**.

Note: This is feasible because the shift cipher has a small key space.

Frequency-Based Cryptanalysis of Shift Ciphers

- Instead of brute-force, we can use **letter frequency** in ciphertext.
- Common English letters and their approximate frequencies:

E(13%), T(9%), A(8%), O(8%), I(7%), N(7%), S(7%), H(6%), R(6%)

- **Steps:**

- ① Compute frequency of each letter in ciphertext.
 - ② Assume the most frequent ciphertext letter encodes 'E'.
 - ③ Compute the shift k from this assumption.
 - ④ Decrypt using shift $-k$. If readable, hypothesis likely correct.
 - ⑤ Otherwise, assume the ciphertext letter encodes 'T', then 'A', etc.
- This intelligent guesswork allows effective decryption without trying all shifts.

Example: Breaking a Shift Cipher using Frequency Analysis

Ciphertext: ZNK KGXRE HOXJ MKZY ZNK CUXS

Step 1: Frequency Count

- Most common letter in ciphertext: K
- Hypothesis: $E \rightarrow K$ implies shift $k = 6$

Step 2: Decrypt with Shift -6

ZNK KGXRE HOXJ MKZY ZNK CUXS $\Rightarrow$ THE EARLY BIRD GETS THE WORM

Conclusion: The plaintext makes sense. The correct shift is $k = 6$.

Transposition Cipher: A Simple Block Cipher

Definition: A *transposition cipher* is a type of block cipher where a permutation is applied to reorder characters in a block.

Encryption Process:

- Choose a block size m and a permutation σ of $\{1, 2, \dots, m\}$.
- Split the message into blocks of m letters.
- For each block $p_1 p_2 \cdots p_m$, generate ciphertext block:

$$c_1 c_2 \cdots c_m = p_{\sigma(1)} p_{\sigma(2)} \cdots p_{\sigma(m)}$$

- If message length $\not\equiv 0 \pmod{m}$, pad with random letters.

Decryption:

- Use σ^{-1} , the inverse permutation.
- Rearrange $c_1 c_2 \cdots c_m$ back to original block.

Example: Transposition Cipher

Given: Block size $m = 4$, permutation $\sigma = (3, 1, 4, 2)$

Step 1: Message

Message = STOP THIEF (add one random letter to make 12 characters)

Padded Message: STOPTHIEFXYZ

Step 2: Break into blocks:

STOP, THIE, FXYZ

Step 3: Apply permutation $\sigma = (3, 1, 4, 2)$

Encrypt each block $p_1p_2p_3p_4 \rightarrow p_{\sigma(1)}p_{\sigma(2)}p_{\sigma(3)}p_{\sigma(4)}$

STOP $\rightarrow$ O S P T

THIE $\rightarrow$ I T E H

FXYZ $\rightarrow$ Y F Z X

Ciphertext: OSPT ITEH YFZX

Decryption: Transposition Cipher

Given: Ciphertext = OSPT ITEH YFZX,

Block size $m = 4$, permutation $\sigma = (3, 1, 4, 2)$

Step 1: Inverse permutation

Find σ^{-1} such that $\sigma(i) = j \Rightarrow \sigma^{-1}(j) = i$

$$\sigma = (3, 1, 4, 2) \Rightarrow \sigma^{-1} = (2, 4, 1, 3)$$

Step 2: Apply σ^{-1} to each block:

OSPT $\rightarrow$ S T O P

ITEH $\rightarrow$ T H I E

YFZX $\rightarrow$ F X Y Z

Decrypted Message: STOP THIEF XYZ

Private Key vs Public Key Cryptography

Private Key Cryptography (Symmetric)

- Same key used for encryption and decryption.
- Both sender and receiver must securely share the key.
- Fast and efficient for large data.
- Example: AES, DES

Example:

- Key: $K = \text{secret}$
- Message: HELLO
- Encrypted: XMCKL
- Decrypted back to: HELLO

Public Key Cryptography (Asymmetric)

- Two keys: Public key for encryption, private key for decryption.
- Public key is shared openly; private key is kept secret.
- Useful for secure key exchange and digital signatures.
- Example: RSA, ECC

Example:

- Alice's Public Key: P_A
- Bob encrypts: $C = E_{P_A}(\text{HELLO})$
- Alice decrypts: $\text{HELLO} = D_{S_A}(C)$, where S_A is Alice's private key

What is a Digital Signature?

A **digital signature** is a cryptographic technique used to:

- Authenticate the sender of a message.
- Ensure the **integrity** of the message.
- Provide **non-repudiation** (the sender cannot deny sending it).

Core Mechanism (Public-Key Crypto)

- The sender signs the message using their **private key**.
- The receiver verifies the signature using the sender's **public key**.

Example Scenario

- Alice wants to send a signed message to Bob.
- She computes a hash of the message and encrypts it using her private key.
- This encrypted hash is the **digital signature**.
- Bob receives the message and the signature.
- He uses Alice's **public key** to decrypt the signature and compare it with the hash of the received message.

Notation Example

- Message: $M = \text{"Hello"}$
- Signature: $S = E_{\text{private}}(H(M))$
- Verification: $H(M) \stackrel{?}{=} D_{\text{public}}(S)$

Homomorphic Encryption: Background and Concept

What is Homomorphic Encryption?

Homomorphic Encryption (HE) is an encryption scheme that allows computation on ciphertexts, producing an encrypted result that, when decrypted, matches the result of operations performed on the plaintexts.

Motivation and Applications

- **Secure Cloud Computation:** Perform computation on encrypted data without revealing the data.
- **Privacy-Preserving Search:** Enables keyword searches on encrypted databases.
- **Secure Voting, Statistics, and Machine Learning.**

Homomorphic Encryption: Background and Concept

What is Homomorphic Encryption?

Homomorphic Encryption (HE) is an encryption scheme that allows computation on ciphertexts, producing an encrypted result that, when decrypted, matches the result of operations performed on the plaintexts.

Motivation and Applications

- **Secure Cloud Computation:** Perform computation on encrypted data without revealing the data.
- **Privacy-Preserving Search:** Enables keyword searches on encrypted databases.
- **Secure Voting, Statistics, and Machine Learning.**

Basic Idea

Given encryption function E , decryption function D , and operation $\circ$:

$$D(E(a) \circ E(b)) = a * b$$

Homomorphic Encryption: Example

Simple Additive Homomorphism Example (Modular Arithmetic)

Let:

- Plaintext addition: $a + b$
- Encryption: $E_k(m) = m + k \pmod{n}$
- Decryption: $D_k(c) = c - k \pmod{n}$

Homomorphic Encryption: Example

Simple Additive Homomorphism Example (Modular Arithmetic)

Let:

- Plaintext addition: $a + b$
- Encryption: $E_k(m) = m + k \pmod{n}$
- Decryption: $D_k(c) = c - k \pmod{n}$

Then:

$$E_k(a) + E_k(b) = (a + k) + (b + k) = a + b + 2k$$

$$D_k(E_k(a) + E_k(b) - k) = a + b$$

Homomorphic Encryption: Example

Simple Additive Homomorphism Example (Modular Arithmetic)

Let:

- Plaintext addition: $a + b$
- Encryption: $E_k(m) = m + k \pmod{n}$
- Decryption: $D_k(c) = c - k \pmod{n}$

Then:

$$E_k(a) + E_k(b) = (a + k) + (b + k) = a + b + 2k$$

$$D_k(E_k(a) + E_k(b) - k) = a + b$$

Implication

The encryption preserves the structure of addition:

$$D(E(a) + E(b) - k) = a + b$$

This is an example of partial homomorphism (additive).

1. Cryptography

- RSA encryption relies on modular exponentiation.
- Example: Encrypt m using $c \equiv m^e \pmod{n}$

1. Cryptography

- RSA encryption relies on modular exponentiation.
- Example: Encrypt m using $c \equiv m^e \pmod{n}$

2. Check Digits in IDs (e.g., ISBN, Credit Cards)

- Uses modular arithmetic to detect input errors.
- Example (ISBN-10):

$$1 \cdot d_1 + 2 \cdot d_2 + \cdots + 10 \cdot d_{10} \equiv 0 \pmod{11}$$

3. Clock Arithmetic (Modular Timekeeping)

- Time follows mod 12 or mod 24 arithmetic.
- Example: What time is 9 hours after 5?

$$(5 + 9) \bmod 12 = 2 \Rightarrow \text{Answer: 2 o'clock}$$

3. Clock Arithmetic (Modular Timekeeping)

- Time follows mod 12 or mod 24 arithmetic.
- Example: What time is 9 hours after 5?

$$(5 + 9) \bmod 12 = 2 \Rightarrow \text{Answer: 2 o'clock}$$

4. Scheduling Problems

- Use Chinese Remainder Theorem to solve periodic tasks.
- Example: Bus A every 3 mins, Bus B every 5 mins. When do both arrive together?

$$\text{Solve } x \equiv 0 \pmod{3}, \quad x \equiv 0 \pmod{5} \Rightarrow x \equiv 0 \pmod{15}$$

Hashing and Modular Arithmetic

- Hash functions map data to a fixed-size value (hash code).
- Congruence helps in designing simple and efficient hash functions.
- Used in data retrieval (hash tables), file checks, cryptographic signatures.

Hashing and Modular Arithmetic

- Hash functions map data to a fixed-size value (hash code).
- Congruence helps in designing simple and efficient hash functions.
- Used in data retrieval (hash tables), file checks, cryptographic signatures.

Example: Simple Hash Function

- Suppose we have 10 buckets (indexed 0–9), and want to place record ID = 1234.
- Use the hash function: $h(k) = k \bmod 10$

$$h(1234) = 1234 \bmod 10 = 4$$

- So, store the record in bucket 4.